Preguntas de análisis

Cada respuesta se acompaña de su evidencia en el **código** (fragmento y ubicación en el repositorio) y de su evidencia en **ejecución** (extracto de los logs del documento Evidencias de las pruebas, generados sobre el stack de Docker Compose). Las líneas `[BROKER]` y `[REGISTRO]` provienen de la bitácora del Broker (`docker logs uady-broker`).

1. ¿Cómo reduce el Broker el acoplamiento?

El cliente no conoce ni instancia al proveedor; solo envía el nombre del servicio/operación al Broker, que se encarga de localizarlo y enrutar. Así se puede cambiar o sustituir el proveedor sin tocar al cliente.

Evidencia en el código. La única dirección que conoce el Sistema 2 es la del Broker (`BROKER_HOST`, `BROKER_PORT`), y cada solicitud se arma solo con nombres de servicio y operación (`sistema-cliente/.../controller/RegistroVentaController.java:220`):

```
private JsonElement enviar(String servicio, String operacion, JsonObject
parametros) throws Exception {
    Solicitud solicitud = Solicitud.crear(UUID.randomUUID().toString(),
servicio, operacion, parametros);
    Respuesta respuesta = clienteTcp.enviarSolicitud(solicitud);
    ...
}
```

El Broker decide a quién reenviarla consultando su registro (`broker/.../network/ManejadorConexion.java:169`):

```
if ( Servicios.BROKER.equals(solicitud.getNombreServicio()) ) {
    return procesarOperacionInterna(solicitud);
}
return enrutador.enrutar(solicitud, registro);
```

Evidencia en ejecución (prueba 8, cambio de proveedor). El mismo comando del cliente lo atiende primero el proveedor original y, tras registrarse un segundo proveedor y apagar el primero, el nuevo, sin cambiar nada en el cliente:

```
[BROKER] 2026-11-14 15:18:50.182 [pool-1-thread-10] id=ev-08a
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 3 ms
[REGISTRO] 2026-11-14 15:18:50.503 [pool-1-thread-12] registrar
gestorTiendas -> host.docker.internal:6001
[BROKER] 2026-11-14 15:18:52.639 [pool-1-thread-3] id=ev-08b
gestorTiendas.listarTiendas -> host.docker.internal:6001 | EXITO | 64 ms
```

2. ¿Qué detalles oculta al cliente?

La ubicación de red del proveedor, su implementación interna, base de datos, generación de QR y demás lógica de negocio. El cliente solo ve el contrato JSON de solicitud/respuesta.

Evidencia en el código. El Sistema 1 arma la respuesta de `listarInventario` campo por campo y deja fuera la ruta interna del PNG del QR (`sistema-centralizado/.../network/ManejadoresConsulta.java:77`):

```
// No incluye la ruta del QR, que es interna al contenedor del Sistema 1.
fila.addProperty("numeroSerie", unidad.numeroSerie());
fila.addProperty("idProducto", unidad.idProducto());
fila.addProperty("nombreProducto",
nombres.getOrDefault(unidad.idProducto(), "Producto desconocido"));
fila.addProperty("estado", unidad.estado().name());
```

Evidencia en ejecución (prueba 2, localización de un servicio). El cliente solo envía y recibe el contrato JSON; la dirección del proveedor (`sistema-centralizado:6000`) aparece únicamente en la bitácora del Broker:

```
CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-02",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  ...
}
BROKER -> CLIENTE (105 ms)
{
  "idSolicitud": "ev-02",
```

```
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
    ...

[BROKER] 2026-11-14 15:18:13.496 [pool-1-thread-6] id=ev-02
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 90 ms
```

3. ¿Qué sucede si el Broker deja de funcionar?

Se interrumpe la integración distribuida: las solicitudes no se enrutan y el cliente recibe un error de conexión. Cada sistema puede seguir funcionando localmente, pero sin comunicación entre ellos. Además, como el registro de servicios vive en memoria, al reiniciar el Broker los servicios aparecen como desconocidos hasta que el proveedor se vuelve a registrar.

Evidencia en el código. El registro es un mapa en memoria del proceso del Broker (`broker/.../registry/RegistroServicios.java:23`):

```
private final ConcurrentHashMap<String, Proveedor> servicios = new
ConcurrentHashMap<>();
```

El Sistema 1 se registra solo al arrancar, en un hilo de fondo (`sistema-centralizado/.../Main.java:69`):

```
RegistradorBroker registrador = new RegistradorBroker(
        configuracion, RegistradorBroker.serviciosDelSistema());
Thread hiloRegistro = registrador.iniciarEnSegundoPlano();
```

Evidencia en ejecución (prueba 12, Broker caído). Con el Broker detenido el cliente no obtiene respuesta aunque el Sistema 1 sigue vivo; al reiniciarlo, el registro está vacío:

```
SIN RESPUESTA (13 ms): ConnectException: Connection refused: getsockopt

## Paso 3: el Sistema 1 sigue en ejecucion (docker ps)
```

```
uady-sistema-centralizado | Up 3 seconds

## Paso 5: el Broker volvio, pero su registro esta vacio
[BROKER] 2026-11-14 15:19:06.787 [pool-1-thread-2] id=ev-12b
gestorTiendas.listarTiendas -> (sin proveedor) | ERROR
SERVICIO_DESCONOCIDO: Servicio desconocido: gestorTiendas | 46 ms

## Paso 7: tras reiniciar el Sistema 1, el servicio vuelve a responder
[BROKER] 2026-11-14 15:19:09.278 [pool-1-thread-6] id=ev-12c
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 88 ms
```

4. ¿El Broker constituye un punto único de falla?

Sí, en esta arquitectura simple es el punto único de falla para la comunicación entre cliente y proveedor.

Evidencia en el código. El cliente tiene configurada una sola dirección de Broker y no hay otra ruta hacia el proveedor (`sistema-cliente/.../model/ClienteTCP.java:56`):

```
try (Socket socket = new Socket()) {
    socket.connect(new java.net.InetSocketAddress(host, puerto),
timeoutMs);
```

Evidencia en ejecución (prueba 12). Es la misma prueba de la pregunta 3: con un solo Broker detenido, la solicitud falla aunque MariaDB y el Sistema 1 estén en ejecución:

```
SIN RESPUESTA (13 ms): ConnectException: Connection refused: getsockopt
Resumen: el cliente no pudo comunicarse con el Broker

uady-sistema-centralizado | Up 3 seconds
uady-mariadb | Up 22 seconds (healthy)
```

5. ¿Cómo se podría mejorar su disponibilidad?

Con réplicas del Broker, failover, balanceo de carga, health checks, reconexión automática y reintentos limitados. También ayudan timeouts bien configurados y registro redundante.

De estas medidas, el proyecto ya implementa **timeouts separados de conexión y lectura**, **reintentos limitados** y la **marca de proveedor no disponible** (deja de enviarle solicitudes durante un tiempo de enfriamiento). Las réplicas del Broker, el balanceo y el re-registro periódico quedan como mejoras.

Evidencia en el código. Ciclo de reintentos del enrutador y marca de fallo (`broker/.../routing/Enrutador.java:158`):

```
if (!registro.estaDisponible(solicitud.getNombreServicio())) {
    return Respuesta.error(solicitud.getIdSolicitud(),
CodigoError.PROVEEDOR_NO_REGISTRADO,
            "Proveedor no disponible: " + solicitud.getNombreServicio());
}
...
for (int intento = 1; intento <= intentosTotales; intento++) {
    try (Socket socket = new Socket()) {
        Respuesta respuesta = redireccionarSolicitud(solicitud, socket,
proveedor);
        registro.marcarExito(solicitud.getNombreServicio());
        return respuesta;
    } catch (SocketTimeoutException e) { ... }
    ...
}
registro.marcarFallo(servicio);
```

Los valores se configuran por variables de entorno (`BROKER_TIMEOUT_CONEXION_MS`, `BROKER_TIMEOUT_LECTURA_MS`, `BROKER_REINTENTOS`, `BROKER_FALLOS_PARA_INDISPONIBLE`, `BROKER_INDISPONIBLE_MS`) en `broker/.../util/ConfiguracionBroker.java`.

Evidencia en ejecución (pruebas 1 y 9). Configuración con la que arranca el Broker, reintentos ante un proveedor apagado y marca de no disponible:

```
Broker iniciado en el puerto 5000 con pool de 16 hilos (timeout conexion
2000 ms, timeout lectura 5000 ms, reintentos 2, fallos para no disponible
3, enfriamiento 30000 ms)

[BROKER] 2026-11-14 15:18:54.065 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | intento 1/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:54.267 [pool-1-thread-4] id=ev-09a
```

```
inventario.listarInventario -> host.docker.internal:6001 | intento 2/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:54.469 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | intento 3/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[REGISTRO] 2026-11-14 15:18:56.645 [pool-1-thread-7] inventario marcado
como NO DISPONIBLE tras 3 fallos
Resumen: estado=ERROR, codigo=PROVEEDOR_NO_REGISTRADO, mensaje="Proveedor
no disponible: inventario"
```

6. ¿Qué atributo mejora al registrar servicios dinámicamente?

La flexibilidad (y con ella la mantenibilidad/escalabilidad): se pueden agregar o sustituir proveedores en caliente sin modificar al cliente.

Evidencia en el código. El registro se hace en tiempo de ejecución con la operación `broker.registrarProveedor` (`broker/.../network/ManejadorConexion.java:210` y `broker/.../registry/RegistroServicios.java:72`):

```
// ManejadorConexion.registrarDesdeSolicitud
Proveedor proveedor = GsonFactory.DEFAULT.fromJson(params,
Proveedor.class);
registro.registrar(proveedor);

// RegistroServicios.registrar
servicios.put(proveedor.servicio(), proveedor);
salud.remove(proveedor.servicio());
```

Evidencia en ejecución (pruebas 1 y 8). Registro de los tres servicios al arrancar y registro en caliente de un segundo proveedor sin detener el Broker:

```
[REGISTRO] 2026-11-14 15:18:04.923 [pool-1-thread-2] registrar
gestorTiendas -> sistema-centralizado:6000
[REGISTRO] 2026-11-14 15:18:04.946 [pool-1-thread-3] registrar inventario
-> sistema-centralizado:6000
[REGISTRO] 2026-11-14 15:18:04.948 [pool-1-thread-4] registrar
gestorPrecios -> sistema-centralizado:6000
```

```
Proveedor registrado en el broker localhost:5000 como
host.docker.internal:6001 (gestorTiendas, inventario, gestorPrecios)
[REGISTRO] 2026-11-14 15:18:50.503 [pool-1-thread-12] registrar
gestorTiendas -> host.docker.internal:6001
```

7. ¿Qué costo introduce el Broker en rendimiento?

Un salto de red extra, serialización/deserialización JSON, enrutamiento y overhead de sockets/hilos. Puede volverse cuello de botella si no se dimensiona bien.

Evidencia en el código. Por cada solicitud, el Broker abre una conexión adicional hacia el proveedor y vuelve a serializar y deserializar el mensaje (`broker/.../routing/Enrutador.java:224`):

```
socket.connect(new InetSocketAddress(proveedor.host(), proveedor.puerto()),
timeoutConexionMillis);
...
salida.println(GsonFactory.DEFAULT.toJson(solicitud));
...
String respuestaJson = entrada.readLine();
...
Respuesta respuesta = GsonFactory.DEFAULT.fromJson(respuestaJson,
Respuesta.class);
```

Evidencia en ejecución (prueba 14). La misma operación, 100 veces directo al Sistema 1 y 100 veces a través del Broker:

```
Camino                              promedio    mediana     minimo
maximo
Directo al Sistema 1 (:6000)          3.53 ms     3.19 ms     2.73 ms
11.46 ms
A traves del Broker (:5000)           4.45 ms     4.07 ms     3.35 ms
11.27 ms

Resumen: el Broker agrega en promedio 0.92 ms por solicitud (26.2% mas que
la llamada directa).
```

8. ¿Dónde se realiza el marshalling y unmarshalling?

En los tres componentes: el cliente serializa al enviar y parsea al recibir; el Broker convierte mensajes en tránsito; el proveedor hace lo propio. Se apoya en Gson en cada sistema.

Evidencia en el código. Los tres usan la misma `GsonFactory` del módulo `common`:

Componente	Ubicación	Marshalling / unmarshalling
Cliente (Sistema 2)	`sistema-cliente/.../model/ClienteTCP.java`	`gson.toJson(solicitud)` y `gson.fromJson(jsonRespuesta, Respuesta.class)`
Broker	`broker/.../util/JsonBroker.java:24` y `:37`	`fromJson(json, Solicitud.class)` y `toJson(respuesta)`
Broker → proveedor	`broker/.../routing/Enrutador.java:231` y `:239`	`toJson(solicitud)` y `fromJson(respuestaJson, Respuesta.class)`
Proveedor (Sistema 1)	`sistema-centralizado/.../network/JsonServicio.java:31` y `:45`	`fromJson(json, Solicitud.class)` y `toJson(respuesta)`

Evidencia en ejecución (prueba 5). El mensaje viaja como una línea JSON. El cliente lo serializa con su `fechaHora` local y la respuesta llega con la `fechaHora` del contenedor (UTC), lo que muestra que se deserializó y volvió a serializar en otro proceso:

```
CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-05",
  "nombreServicio": "servicioFantasma",
  "nombreOperacion": "listar",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:16.6061441"
```

```
}
BROKER -> CLIENTE (14 ms)
{
  "idSolicitud": "ev-05",
  "estado": "ERROR",
  "codigoError": "SERVICIO_DESCONOCIDO",
  "mensajeError": "Servicio desconocido: servicioFantasma",
  "fechaHora": "2026-11-14T15:18:16.638050827"
}
```

9. ¿Cómo se gestionarían solicitudes duplicadas?

Con un id único por solicitud (idempotency key). El Broker/proveedor detectan repetidos por ese id, evitan reprocesar y pueden devolver la respuesta ya generada.

En la implementación actual cada solicitud ya lleva un `idSolicitud` único (UUID), pero todavía no se usa para detectar repetidos. Lo que evita la duplicación de una venta es la **transacción con bloqueo pesimista** del Sistema 1: la segunda solicitud encuentra la unidad ya `VENDIDO` y se rechaza. La idempotency key queda como mejora para devolver la respuesta original en lugar de un error.

Evidencia en el código (`sistema-centralizado/.../repository/VentaRepository.java:24` y `:90`):

```
"SELECT estado FROM mercancia WHERE numero_serie = ? FOR UPDATE";
...
conexion.setAutoCommit(false);
try {
    validarMercanciaDisponible(conexion, numeroSerie);  // "La mercancía ya
fue vendida"
    ...
    insertarVenta(conexion, numeroSerie, idTienda, idUsuario);
    actualizarEstado(conexion, numeroSerie);
    conexion.commit();
```

Evidencia en ejecución (prueba 13). La misma venta enviada dos veces: la primera se registra, la segunda se rechaza y en la base de datos queda una sola venta:

```
[BROKER] 2026-11-14 15:18:18.997 [pool-1-thread-16] id=ev-13a
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | EXITO |
```

```
10 ms
[BROKER] 2026-11-14 15:18:19.550 [pool-1-thread-1] id=ev-13b
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | ERROR
PARAMETROS_INVALIDOS: La mercancía ya fue vendida | 6 ms

Ventas registradas para SN-UADY-0005 en la base de datos:
ventas
1
```

10. ¿Cómo se monitorearía el Broker?

Con logs por request id, métricas de latencia/throughput/conexiones/timeouts, health checks, alertas y monitoreo de recursos (CPU, memoria, hilos).

El proyecto ya cuenta con una **bitácora en consola** que registra cada solicitud con su id, operación, proveedor destino, resultado y tiempo de atención, además de los cambios en el registro y los intentos fallidos. En Docker Compose, el Broker también tiene un health check (`nc -z 127.0.0.1 $BROKER_PORT`).

Evidencia en el código (`broker/.../util/BitacoraBroker.java:86`):

```
public void registrarSolicitud(Solicitud solicitud, Respuesta respuesta,
String proveedor, long millis) {
    ...
    escribir(PREFIJO_SOLICITUD, "id=" + id + " " + operacion
            + " -> " + (proveedor == null ? "(sin proveedor)" : proveedor)
            + " | " + resultado(respuesta)
            + " | " + millis + " ms");
}
```

Evidencia en ejecución (prueba 11, 20 clientes simultáneos). La bitácora permite seguir cada solicitud por su id, el hilo que la atendió y su latencia:

```
ev-11-01  inventario.listarInventario    -> idRespuesta=ev-11-01
(coincide)  EXITO  70 ms
ev-11-02  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-02
(coincide)  EXITO  70 ms
...
Resumen: 20/20 clientes recibieron EXITO con el id de su propia solicitud;
tiempo total 83 ms
```

```
[BROKER] 2026-11-14 15:18:20.337 [pool-1-thread-1] id=ev-11-20
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 11 ms
[BROKER] 2026-11-14 15:18:20.338 [pool-1-thread-11] id=ev-11-02
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 12 ms
[BROKER] 2026-11-14 15:18:20.338 [pool-1-thread-9] id=ev-11-16
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 13 ms
```